

TP — Clickjacking: The Button You Didn't Click

Imane Fouad, UM6P

Scenario

Alice regularly uses the university **Student Portal**. After logging in, her browser keeps her authenticated session active. Like many students, Alice also happens to be a big fan of small online games.

Eve wants to take advantage of both facts: Alice is already authenticated, and she can hardly resist a new game. Instead of trying to steal Alice's password, Eve has another idea: **what if Alice could be tricked into interacting with the Student Portal without realizing it?**

Eve has hired **you** to build the proof of concept. Your mission is to create an innocent-looking mini-game, secretly place the Student Portal behind its interface, and investigate whether Alice's clicks can be redirected to the hidden portal. For now, your job is to make Eve's experiment work. *Later on, you might start feeling a little guilty about what you did to Alice... and perhaps decide to switch sides and build a defense for her.*

Note

- All pages in this TP are created locally for educational purposes. Do not apply these techniques to systems without explicit authorization.
- Throughout the TP, document your progress carefully: After completing each part, take a **screenshot** showing the result, push your code to your Git repository, add short explanations or observations to your report when necessary.

1 HTML Knowledge You Need

Notice

Already familiar with basic HTML?

If you have already created a simple HTML page and understand the roles of `<html>`, `<head>`, `<body>`, headings, paragraphs, and buttons, you may quickly review this section and continue to the next part.

For this TP, you only need a few basic HTML elements:

Tag	Purpose
<code><h1></code>	Main heading
<code><p></code>	Paragraph
<code><input></code>	Input field
<code><button></code>	Clickable button
<code>
</code>	Line break
<code><script></code>	JavaScript code
<code><style></code>	CSS rules

Create Your First HTML File

Using your preferred text or code editor, create a new file named:

```
hello.html
```

Save it inside the TP-Clickjacking directory.

Enter the following code:

```

<!DOCTYPE html>

<html>

<head>
  <title>My First Website</title>
</head>

<body>

'''
<h1>Welcome to My First Website</h1>

<p>
  This is my first HTML page.
</p>

<button onclick="showMessage()">
  Click Me
</button>

<script>
  function showMessage() {
    alert("Hello from my first website!");
  }
</script>
'''

</body>

</html>

```

Save the file and double-click it. Congrats you've created your first page.

Notice

Reference: When needed, consult the **Quick Reference Tables** in the Appendix for the HTML elements, CSS properties, and JavaScript instructions used throughout this TP.

2 Create the Student Portal

Goal: create a legitimate local website where a student enters a name and password and clicks a login button.

Create a file named `portal.html`. The page must contain:

- A title: **Student Portal**
- A field for the student name
- A password field
- A button labeled **Login**

When the user clicks **Login**, display **You are logged in!**

```

<!DOCTYPE html>
<html>
<head>
  <title>Student Portal</title>

```

```

</head>
<body>
  <h1>Student Portal</h1>

  <!-- Add the name field -->
  <!-- Add the password field -->
  <!-- Add the Login button -->

  <script>
    function login() {
      // Display "You are logged in!"
    }
  </script>
</body>
</html>

```

Your Task

Complete the missing parts yourself, save the file, open it in your browser, and verify that the button works.

3 Create the Decoy Game

Goal: Create a *Catch Me!* game where each click increases the score and moves the button.

Create `game.html`:

```

<!DOCTYPE html>

<html>
<head>
  <title>Catch Me!</title>
  <style>
    #catchButton {
      position: absolute;
      top: 150px;
      left: 200px;
    }
  </style>
</head>

<body>

  <h1>Catch Me!</h1>
  <p>Score: <span id="score">0</span></p>

  <button id="catchButton">Catch Me</button>

</body>
</html>

```

Position the Button: `top` controls the vertical position and `left` the horizontal position.

Your Task

Change `top` and `left`, save, refresh, and observe the result.

Control CSS with JavaScript

JavaScript can select an element and modify its CSS:

Instruction	Purpose
<code>document.getElementById("id")</code>	Selects the button using its <code>id</code> .
<code>button.style.left</code>	Changes the horizontal position.
<code>button.style.top</code>	Changes the vertical position.

Your Task

Using the instructions in the table above, add a JavaScript function to your game page that changes the position of the **Catch Me** button when it is clicked.
Choose the new `top` and `left` values manually and test your function with different positions.

Build the Game Previously, you changed the button position manually. Now, update your game so that the button moves to a **random position** after each click.

Your Task

Create a function `catchMe()` and connect it to the button using `onclick`.
Each click must:

1. increase and display the score;
2. generate new `x` and `y` coordinates;
3. move the button using `.style.left` and `.style.top`.

4 About Iframes

To trick Alice into interacting with the Student Portal while she believes she is playing the game, we need to display the legitimate portal **inside the game page**. This can be achieved using an `iframe`.

In our scenario, `game.html` acts as Eve's decoy page, while `portal.html` is embedded inside it. Alice therefore opens the game, but the legitimate Student Portal is also present within the same browser page.

Your Task

Add the Student Portal to your game using an `iframe`. Keep the `iframe` fully visible first. Align the Game and the Portal, and verify that you can interact with the Login button inside the game page.

5 Hiding the Student Portal

The Student Portal is now embedded inside `game.html`, but Alice can still see it. Eve therefore needs to make the `iframe` transparent while keeping it positioned over the game.

In `game.html`, add the `opacity` property to the CSS rule of your `iframe`.

Try the following values **one at a time**:

```
opacity: 1;
opacity: 0.5;
opacity: 0.1;
opacity: 0;
```

After each change, save `game.html`, refresh the browser, and observe the Student Portal.

Your Task

Your Task

Gradually reduce the `iframe` opacity until the Student Portal is no longer visible.
Then test whether you can still interact with the portal while its `iframe` is transparent.

6 The Clickjacking Moment

For the attack to work, the visible **Catch Me** button and the hidden **Login** button must be aligned at the moment Alice clicks.

Since the game button normally moves randomly, modify the game so that after a chosen score (for example, score 4), it moves to a **predefined position** directly underneath the portal's Login button.

Important: The two buttons must overlap. The iframe containing the Login button must also be positioned above the game button using **z-index**.

Your Task

Your Task

Choose a score at which the attack will occur. At that score:

1. move the **Catch Me** button to a predefined position;
2. position the iframe so that the **Login** button is directly above it;
3. keep the iframe visible while adjusting the alignment;
4. once correctly aligned, make the iframe transparent and test the click.

7 Reveal the Attack

After the clickjacking test, add a button that reveals the hidden Student Portal. This allows you to verify that the portal's Login button was positioned directly above the game button.

Add the following button to `game.html`:

```
<button onclick="revealAttack()"> Reveal Hidden Page </button>
```

Create a JavaScript function that changes the iframe opacity.

Your Task

When **Reveal Hidden Page** is clicked:

1. make the iframe partially visible;

8 Optional Research Challenge – Defend Alice's Portal

After helping Eve build the attack, you decide it may be time to switch sides and help Alice.

Challenge: Investigate how a web application can prevent its pages from being embedded inside unauthorized frames.

Research the following concepts:

- `X-Frame-Options`
- Content Security Policy (CSP)
- The `frame-ancestors` directive

Your Task

Your Task

Assume that the Student Portal should **never** be displayed inside another website.

Based on your research:

1. Identify the possible anti-clickjacking protections.
2. Explain briefly how each protection controls framing.
3. Determine which protection would allow only same-origin framing.
4. Propose an appropriate protection for the Student Portal.
5. How would your answer change if the university needed to embed the Student Portal inside another trusted university application?

A HTML, CSS and JavaScript Quick Reference

Element	Purpose
JavaScript Quick Reference	
<code>document.getElementById("id")</code>	Selects an element using its ID.
<code>document.querySelector("iframe")</code>	Selects the first iframe in the page.
<code>element.style.left</code>	Reads or changes the horizontal position.
<code>element.style.top</code>	Reads or changes the vertical position.
<code>element.style.opacity</code>	Reads or changes the transparency.
<code>element.textContent</code>	Reads or changes displayed text.
<code>let score = 0;</code>	Creates and initializes a variable.
<code>score++;</code>	Increases a value by 1.
<code>Math.random()</code>	Generates a random value between 0 and 1.
<code>Math.floor(value)</code>	Rounds a value down to an integer.
<code>alert("message");</code>	Displays a popup message.
HTML Quick Reference	
<code><h1>...</h1></code>	Main heading.
<code><p>...</p></code>	Paragraph.
<code><input type="text"></code>	Text input field.
<code><input type="password"></code>	Password input field.
<code><button>...</button></code>	Clickable button.
<code>id="..."</code>	Gives an element an identifier.
<code>onclick="function()"</code>	Calls a function when clicked.
<code>...</code>	Holds text that can be modified.
<code><iframe src="..."></code>	Displays another page.
CCS Quick Reference	
<code>position: absolute;</code>	Enables coordinate positioning.
<code>top: 100px;</code>	Vertical position.
<code>left: 100px;</code>	Horizontal position.
<code>width: 500px;</code>	Element width.
<code>height: 300px;</code>	Element height.
<code>z-index: 2;</code>	Controls stacking order.
<code>opacity: 0.5;</code>	Controls transparency (0–1).

Table 1: Reference